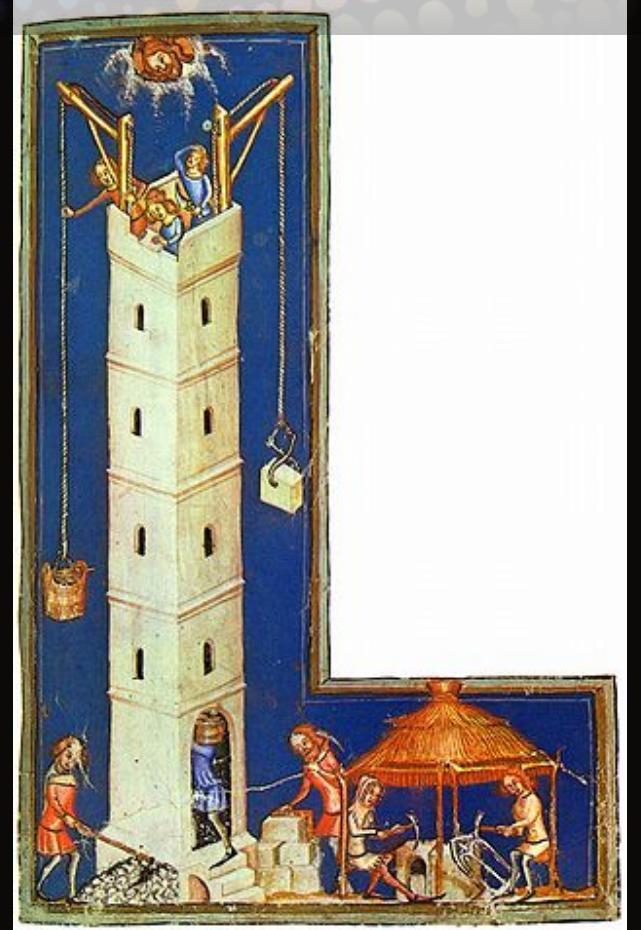


Gen AI Grows Up

Building Production-Ready Agents on the JVM

Rod Johnson, Embabel
GOTO Copenhagen 2025



About Me

- Creator of Spring Framework
- Founder/CEO of SpringSource
- Founder of Embabel
- Developer, entrepreneur, author, investor



Let's Make This Personal

Gen AI is Amazing

- Not just hype: life-changing tools, creative powers
- Very real: Changes how we work every day



ChatGPT

Tools Add Power

Mitigate hallucinations, add capabilities



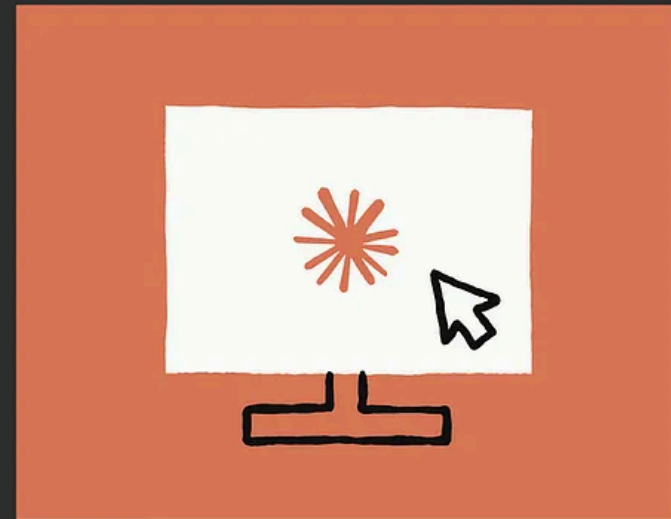
Claude for Desktop Beta

Your AI partner on desktop. Fast, focused, and designed for deep work.

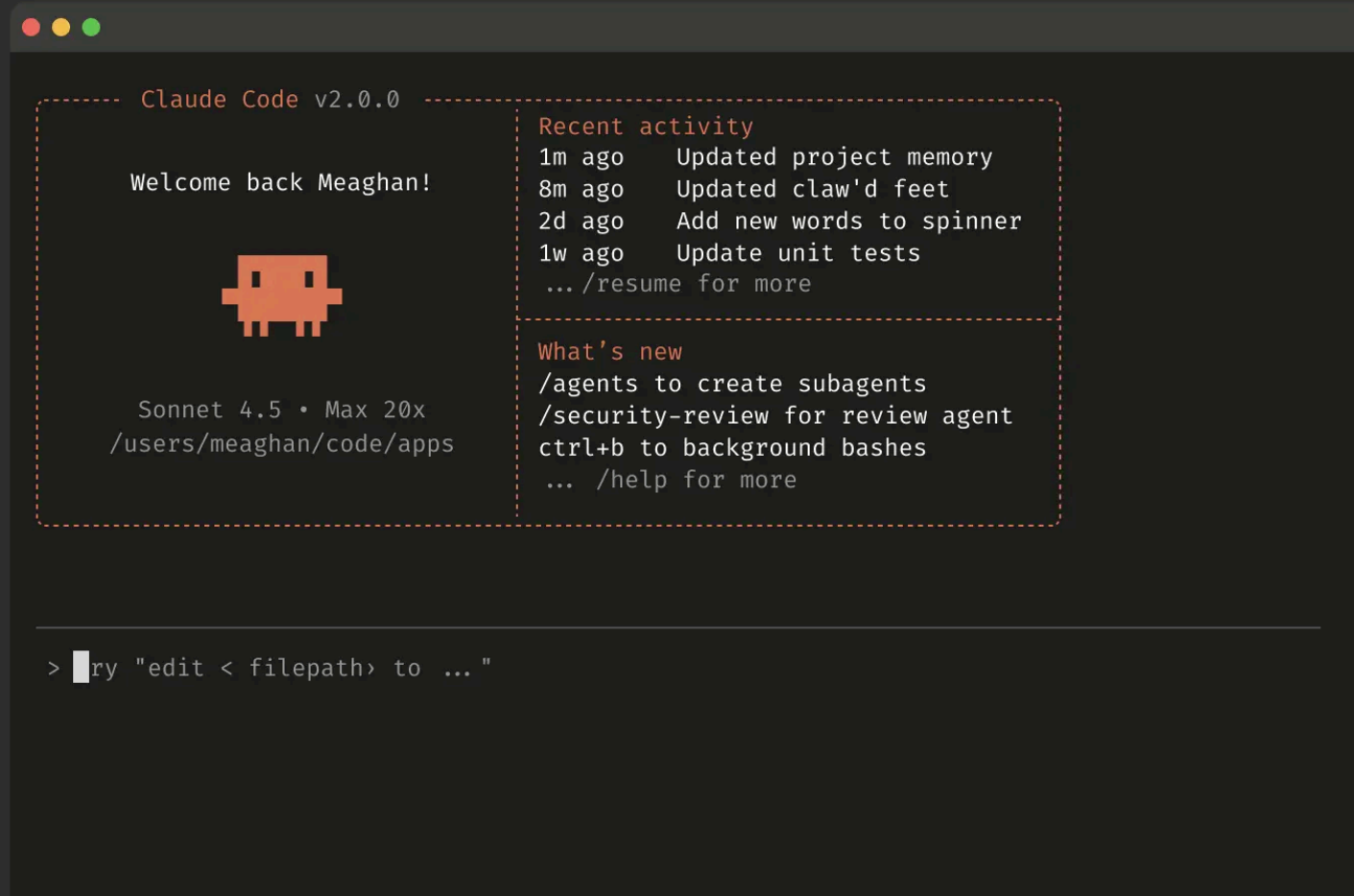
 macOS

 Windows

 Windows (arm64)

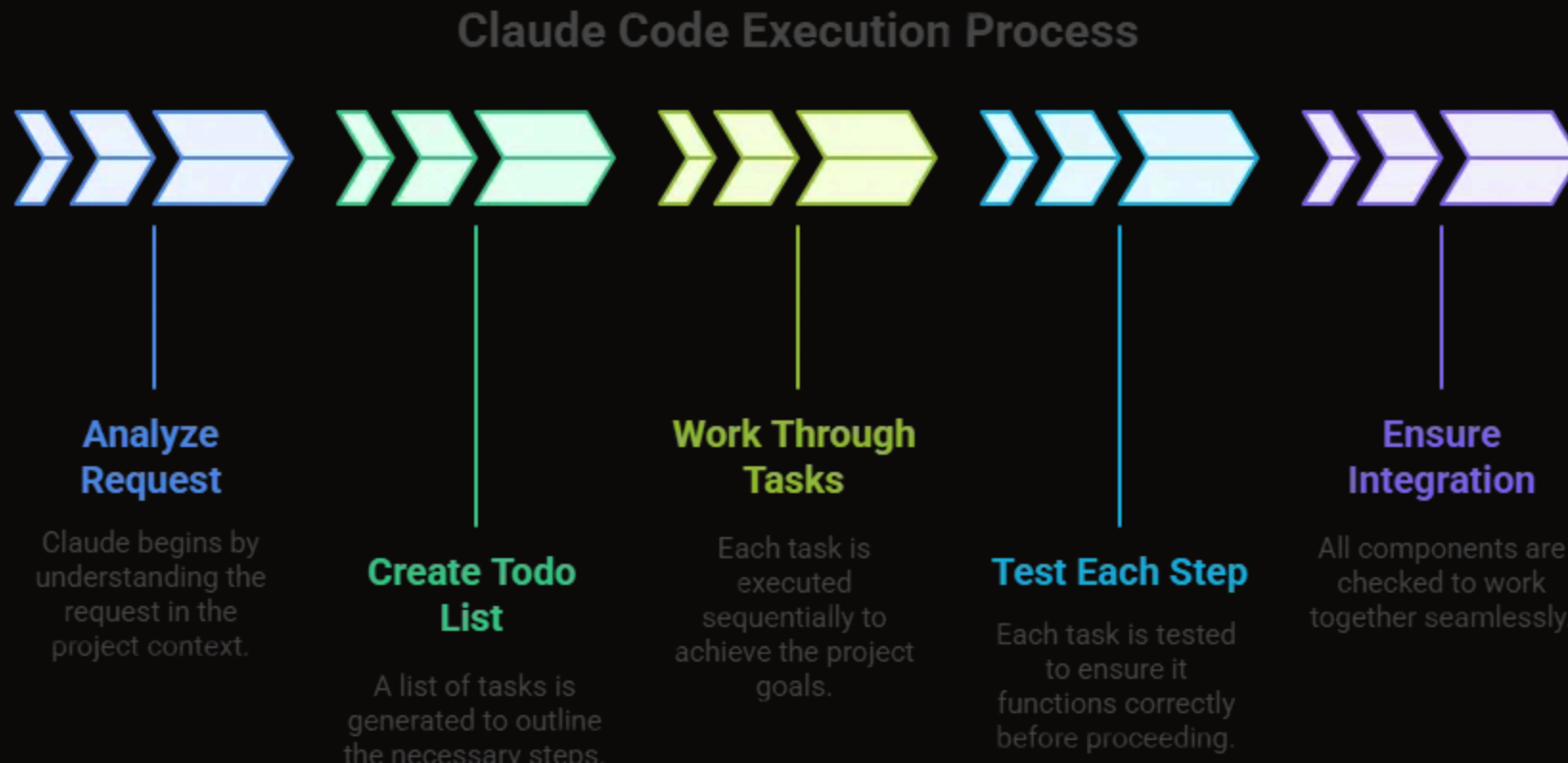


"Personal Assistant" Use Case Has Proven Quite Broad



Powerful but not Predictable is Sometimes OK

Claude Code approach creates dynamic TODO lists and checks them off





BUT...

Most enterprise Gen AI projects are failing spectacularly

Alarming Failure Rate

- **MIT 2025 report:** 95% of Gen AI pilots fail to yield measurable business returns
- \$30-\$40 billion in investments with minimal ROI
- **Gartner:** Over 80% of AI projects will be abandoned post-POC by 2025
- **NTT DATA:** 70-85% of Gen AI deployments miss ROI targets



Why Do Gen AI Projects Fail?

Technology Issues

- Inherently **nondeterministic**
 - Non-determinism once occurred in corner cases
 - It's now unavoidable
- Danger of hallucinations
- Prompts are fragile
- Slow and expensive to run at scale
- Difficult to test and validate

Prompt Engineering: the new Alchemy

- A black art rather than engineering discipline
- Reliance on intuition and experimentation
- Subtle phrasing nuances affect outcomes
- High maintenance overhead
- Sensitive to particular models

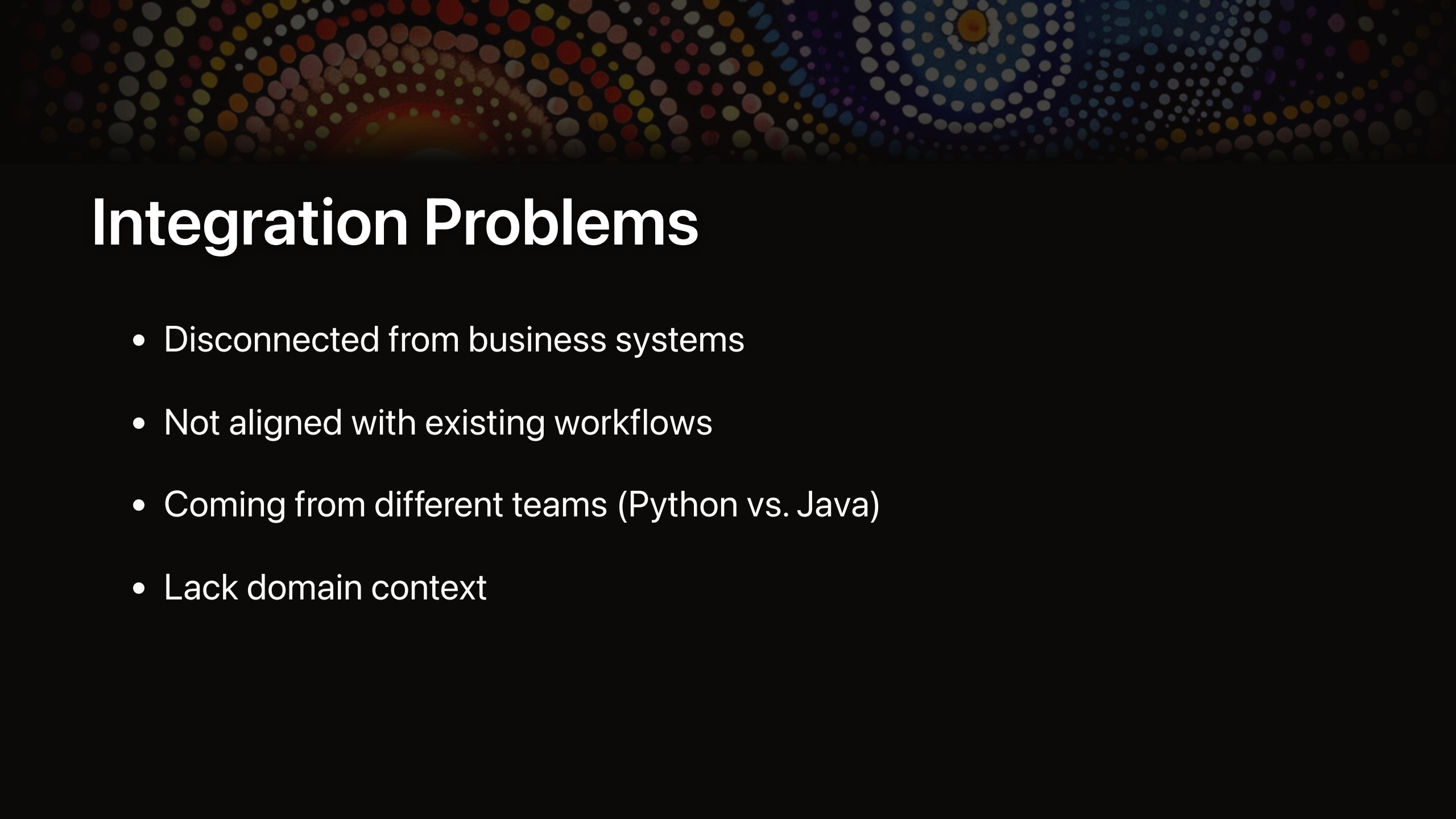
 wizard with cauldron



Approaches that are great for personal assistance don't work for business processes

Depending *entirely* on models and tool calling is too unpredictable

No, MCP doesn't solve all our problems.
We need orchestration

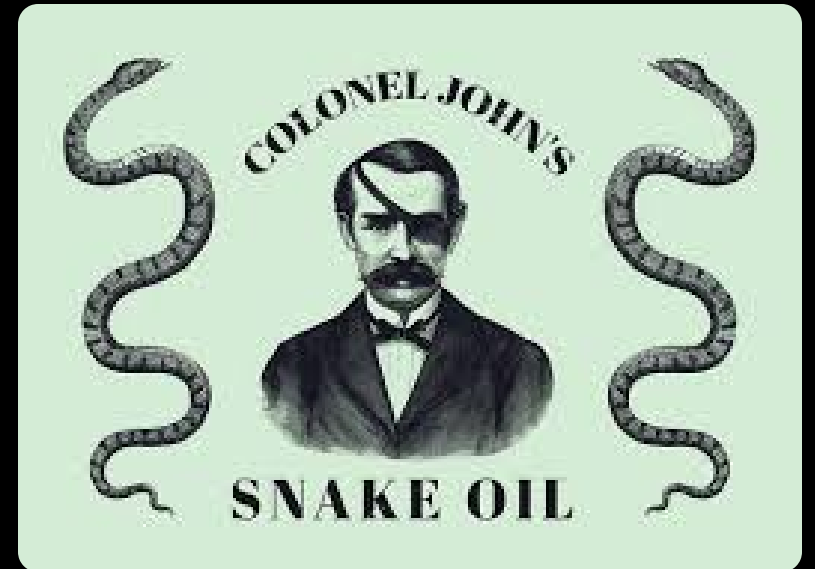


Integration Problems

- Disconnected from business systems
- Not aligned with existing workflows
- Coming from different teams (Python vs. Java)
- Lack domain context

Excessive Vendor Influence

- Marketing hype > practical implementation
- "Snake oil" solutions
- Misaligned priorities
- Lack of domain expertise



Top-Down Mandates

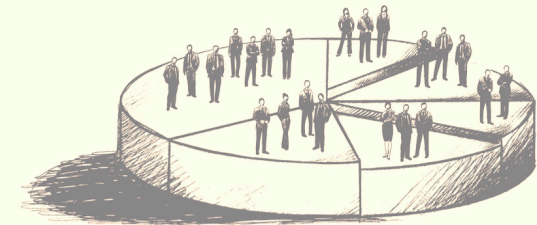
- Guidance from the golf course
 - Grand strategy from executives
 - No clear execution plan
 - We've seen this movie before
- Pressure to deploy without purpose



Organizational Issues

- Central AI teams don't understand the business
- Siloed from domain experts
- Skills gap and resource constraints
- Unclear ownership and responsibility

The cost of organisational silos





How Do We Fix This?

1. Attack Nondeterminism

- Make agents as predictable as possible
 - Break complex tasks into steps with smaller prompts
 - Introduce deterministic planning
 - Add structure to interactions
- Build reliable testing frameworks
- Create guardrails

2. Integrate With What Works

- Don't reinvent the wheel
- Connect to existing systems
- Leverage current domain expertise and coding skills
- Build on proven enterprise infrastructure
- Incremental adoption approach

Bringing Structure to LLM Interactions

- Move beyond "string typing"
- **Domain models** provide structure and behavior
- Type-safe interfaces for I/O
- Clear abstractions and contracts
- Testable and debuggable components

Typical Python use of fragile magic strings:

```
# Initialize the ChatGoogleGenerativeAI module, create and invoke the retriev
llm = ChatGoogleGenerativeAI(model="gemini-pro")

template = """
You are a helpful AI assistant. Answer based on the context provided.
context: {context}
input: {input}
answer:
"""

prompt = PromptTemplate.from_template(template)
```

Domain-Integrated Context Engineering

- Bridges LLMs and business systems
- Adds structure to inputs and outputs
- Domain objects encapsulate data AND behavior
- Bi-directional communication in domain terms
- Integration with existing domain models



What is the Role of Java Developers?

| What I say about Java, applies to .NET

Option 1: Imitate Python Approaches

- Copy Python frameworks in Java
- Accept limitations of those approaches
- Maintain disconnection from business logic
- Build bridges between two worlds

| Doomed to second-class status, accepting being downstream of innovation

Option 2: Build Something Better

- Bring Java/JVM strengths to Gen AI
- Leverage existing domain knowledge
- Apply enterprise architecture experience
- Create frameworks that integrate naturally
- **Lead** rather than follow

We understand business applications: We have the skills



The Stakes are High

Failure to fully benefit from Gen AI is a huge missed opportunity

Neither we as JVM developers or business as a whole can afford the consequences of Option 1

The same is true of .NET: It has a critical role to play

What the JVM Brings to Gen AI

- Maturity and stability
- Strong typing and compile-time safety
- Rich ecosystem of libraries and tools
- Proven reliability at scale
- Excellent concurrency support
- Decades of honing for building enterprise apps



Python vs. JVM for Enterprise AI

- **Python:** Great for data science, scripting, prototyping
- **JVM:** Excels at enterprise-grade applications

Data Science != Gen AI

IF YOU HAVE ZERO ENTERPRISE APPS IN PYTHON, THAT REMAINS THE
RIGHT NUMBER

Time for Java Community to Lead Again

- Spring and other open source revolutionized enterprise Java, saving it from top down failure
- The same factors again apply
 - Open source community collaboration
 - Practical solutions to real problems
 - Learning from existing approaches
 - Adding genuinely new ideas





Introducing Embabel

Embabel: A Framework for Production-Ready Agents

- Addresses key Gen AI failure points directly
- Maximizes determinism
- Emphasizes reliability and testability
- Spring integration for easy adoption
- Modern API with strong typing



Innovation: Deterministic Planning

- **Goal Oriented Action Planning**
(GOAP)
- Predictable, explainable execution
- **Actions** and **Goals** create extensible
system
- Pre and post conditions usually
expressed in terms of domain objects

Embabel Actions and Goals

```
@Action
fun identifyResearchTopics(
    presentationRequest: PresentationRequest,
    ai: Ai
): ResearchTopics =
    config.planner.promptRunner(ai)
        .withRag(config.ragOptions())
        .withReferences(presentationRequest.llmReferences)
        .create(
            """
            Create a list of research topics for a presentation,
            based on the given input:
            ${presentationRequest.brief}
            About the presenter: ${presentationRequest.presenterBio}
            """).trimIndent()
        )
```

@AchievesGoal Annotation

```
@AchievesGoal(  
    description = "Create a presentation based on research reports",  
    export = Export(remote = true),  
)  
@Action  
fun convertToSlides(  
    presentationRequest: PresentationRequest,  
    @RequireNameMatch withIllustrations: SlideDeck,  
    context: OperationContext,  
): FileArtifact {  
    val htmlFile = slideFormatter.createHtmlSlides(  
        directory = context.outputDirectory,  
        markdownFilename = presentationRequest.outputFile,  
    )  
    return FileArtifact(  
        directory = context.outputDirectory,  
        outputFile = htmlFile,  
    )  
}
```

Strong Typing and Tooling

- Compile-time safety
- IDE support with code completion
- Refactoring capabilities
- Clear interfaces between components
- Easier to maintain as projects grow

Java Example: Bank Support Agent

```
@Agent(description = "Customer support agent")
record SupportAgent(CustomerRepository customerRepository) {

    @AchievesGoal(description = "Help bank customer with their query")
    @Action
    SupportOutput supportCustomer(SupportInput supportInput, OperationContext context) {
        var customer = customerRepository.findById(supportInput.customerId());
        if (customer == null) {
            return new SupportOutput("Customer not found with this id", false, 0);
        }
        return context.ai()
            .withLlm(OpenAiModels.GPT_41_MINI)
            .withToolObject(customer)
            .createObject(
                """
                You are a support agent in our bank, give the
                customer support and judge the risk level of their query.
                In some cases, you may need to block their card. In this case, explain why.
                Reply using the customer's name, "%s".
                Currencies are in $.

                Their query: [%s]
                """, formatted(customer.name(), supportInput.query()),
                SupportOutput.class);
    }
}
```

Seamless Code Actions & LLM Invocation

- Mix deterministic code with LLM calls
- Each in its proper role
- Much business logic stays in code
- LLMs handle natural language tasks
- Best of both worlds

Builds on Spring

- Familiar programming model: An @Agent is a bean
- Dependency injection
- Autoconfiguration
- Bean lifecycle management
- Transactional support



Modern Java/Kotlin API

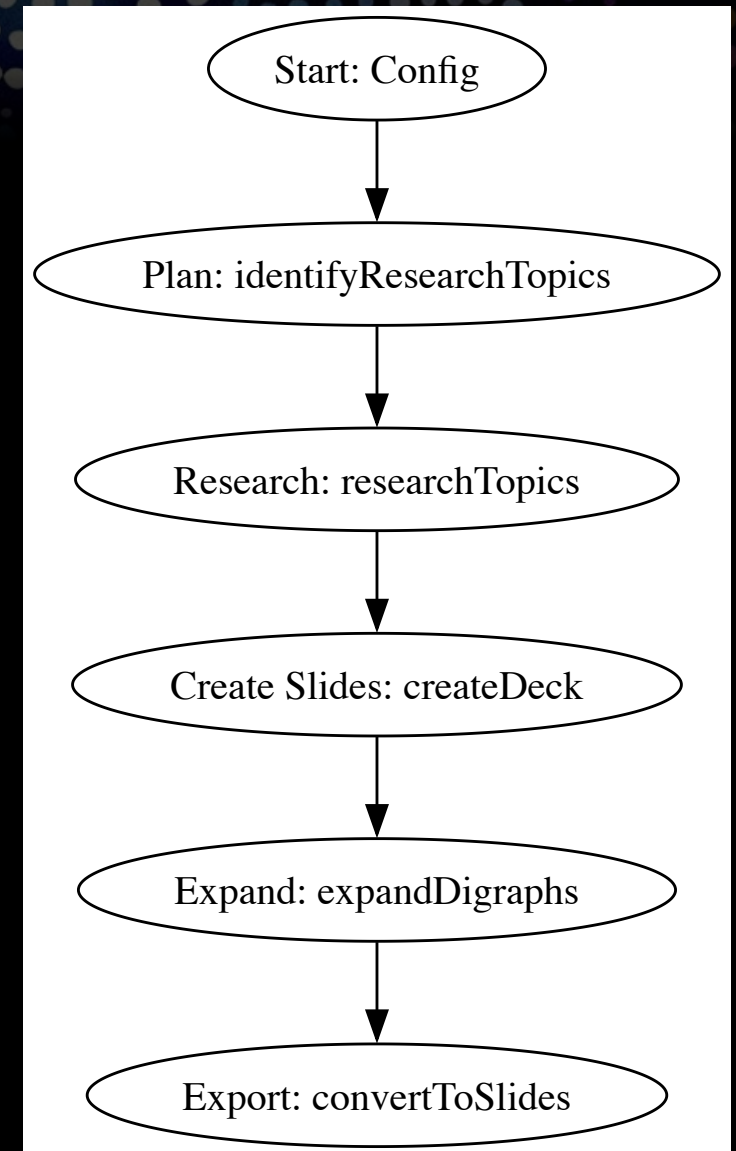
- Fluent interface
- Functional programming support
- Records for immutable data
- Type inference
- Concise, expressive code

Easily Testable Agents

- Unit test individual actions
- Mock LLM responses
- Verify behavior deterministically
- Integration tests for full flows
- CI/CD friendly

Example: This Slide Deck!

- Built with Embabel's **Decker** agent - <https://github.com/embabel/decker>
- Models flow as Actions and Goals
- Illustrates core concepts
- Planning and research actions
- Generation and formatting



Decker Agent Example

```
@Agent(description = "Presentation maker. Build a presentation on a topic")
class Decker(
    private val slideFormatter: SlideFormatter,
    private val filePersister: FilePersister,
    private val config: DeckerConfig,
) {
    @Action
    fun identifyResearchTopics(
        presentationRequest: PresentationRequest,
        ai: Ai
    ): ResearchTopics = /* ... */

    @Action
    fun researchTopics(
        researchTopics: ResearchTopics,
        presentationRequest: PresentationRequest,
        context: OperationContext,
    ): ResearchResult { /* ... */
    }

    @Action
    fun createDeck(
        presentationRequest: PresentationRequest,
        researchComplete: ResearchResult,
        context: OperationContext,
    ): SlideDeck { /* ... */
    }

    // Additional actions...
}
```

Embabel vs. Python Frameworks

Python Crew AI / LangGraph	Embabel
Dynamic typing	Strong typing
Loose integration	Tight domain integration
Finite state machines/fixed steps	Flexible goal-oriented planning
Unstructured text	Domain objects
Separate from existing business apps	Integrated with business
Scripting approach	Sound architecture
Requires new stack	Build on your existing stack

Conclusion

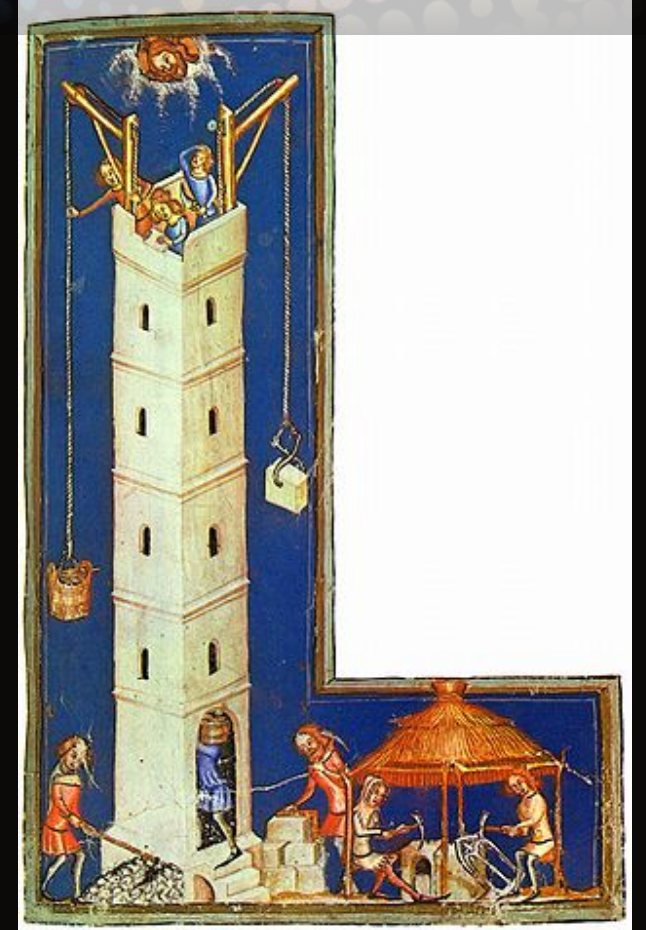
- Gen AI needs to grow up for enterprise adoption
- JVM offers mature, production-ready foundation
- Structure and domain integration are key
- Embabel brings Gen AI and JVM strengths together
- Genuine innovation, not just playing catchup

Start Right Away

Don't wait for a grand transformation strategy

- Put a little Gen AI into your apps today
- Grow towards unlocking full business value
- Create a repo from the Embabel `java-agent-template` to get started
- Join our open source community

What will you automate first?



Thank You!

Rod Johnson

Founder, Embabel

- Twitter: @springrod
- GitHub: <https://github.com/johnsonr>
- Website: <https://embabel.org/>

